

Smart parking lot management system

DBMS Course Project Report



Project Information	Details
Student Name	Mithesh Kosanam
Roll Number	25WU0102130
Course	Database Management Systems (DBMS)
Faculty Name	Dr. Kiran Mayee Adavala
Academic Year	2025–2029
Project Type	DBMS Course Project

Woxsen University

CSE - AIML

2. Abstract

The Smart parking lot management system is a full-stack database application developed to manage customers, vehicles, parking slots, parking sessions, billing, and payments in a structured relational system. The project uses MySQL as the database layer, Node.js and Express.js for backend services, and a React-based frontend for interaction. The system supports real-time viewing, CRUD operations, parking session management, billing, payment tracking, search, and dashboard statistics. Transaction-based parking operations maintain consistency between parking sessions and slot status.

3. Introduction and Problem Statement

3.1 Introduction

Parking facilities generate related data continuously: customers own vehicles, vehicles use parking slots, sessions record entry and exit, bills record charges, and payments settle those bills. Managing these records independently can create duplication and inconsistent status information. This project models the complete flow using a relational database and a connected web interface.

3.2 Problem Statement

A parking operator needs a reliable way to view slot availability, track vehicle movement, maintain customer records, generate billing information, and record payments. The system must also preserve relationships between these records and prevent inconsistent parking states.

4. Objectives and Scope

4.1 Objectives

- Maintain customer and vehicle records in MySQL.
- Track FREE and OCCUPIED parking slots.
- Record vehicle entry and exit through parking sessions.
- Maintain billing and payment information.
- Provide database-backed CRUD operations through the UI.
- Provide dashboard statistics and customer-level search.
- Maintain consistency using relational constraints and transactions.

4.2 Scope

The scope covers a single smart parking database with six core tables and a web interface for operational management. The current system focuses on database-backed parking operations rather than physical sensor or hardware control.

5. Software and Hardware Requirements

Category	Requirement
Operating System	macOS / Windows / Linux
Frontend	React 19, TypeScript, TanStack Start/Router, TanStack Query
Styling & UI	Tailwind CSS v4, Lucide React, Recharts
Backend	Node.js, Express.js
Database	MySQL
DB Connectivity	mysql2
Configuration	dotenv
Browser	Modern Chromium/Firefox/Safari browser
Recommended Hardware	Dual-core processor, 8 GB RAM, 10 GB free storage

The software stack above reflects the project repository. The hardware values are recommended local-development requirements rather than strict application limits.

6. ER Diagram

The ER diagram represents the six core entities and their relationships. It maps customers to vehicles, vehicles to parking sessions, sessions to slots and bills, and bills to payments.

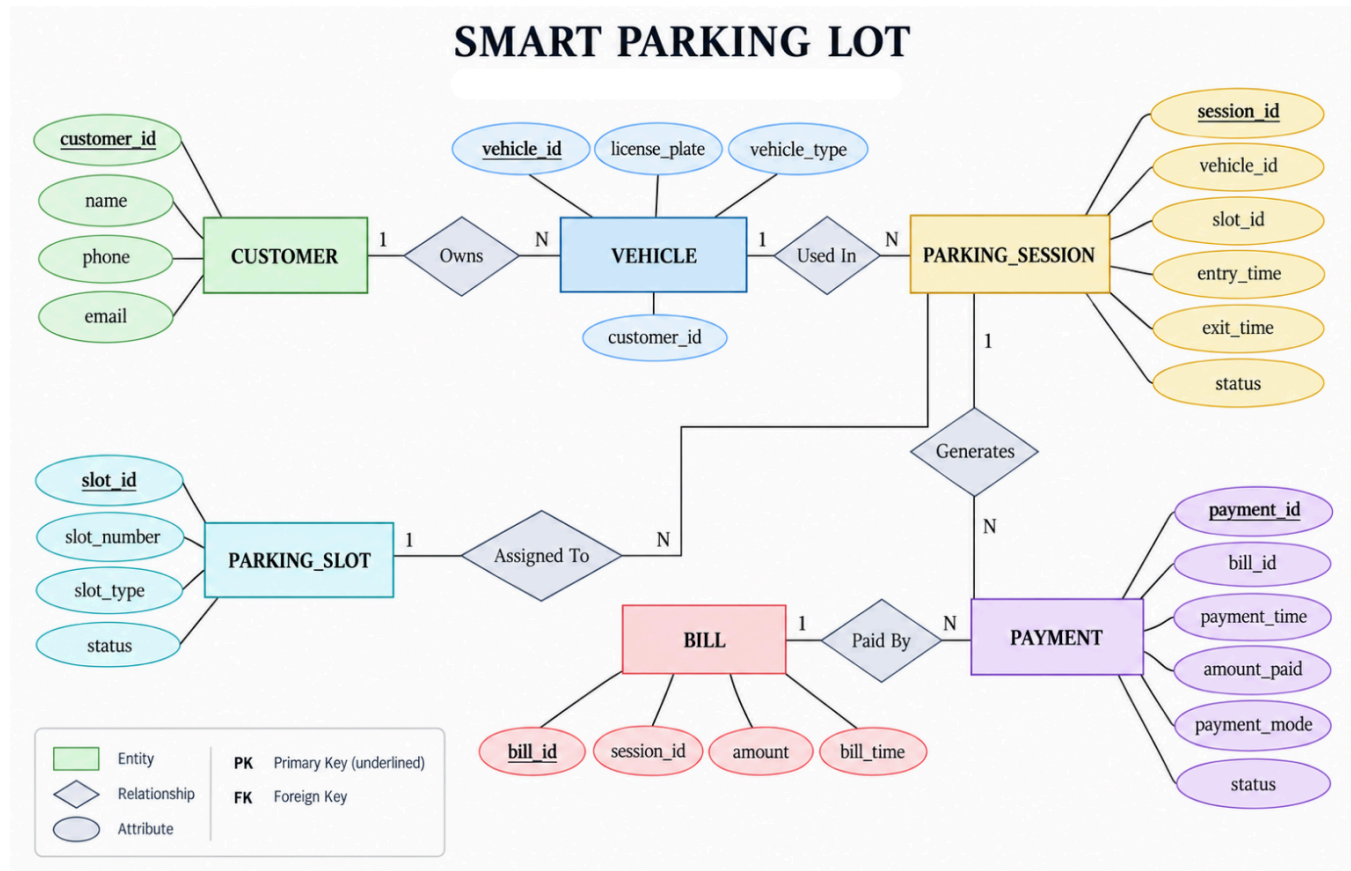


Figure 1. ER diagram placeholder — insert the finalized ER diagram here.

7. Relational Schema and Normalization

7.1 Relational Schema

Relation	Primary Key	Foreign Keys
CUSTOMER	customer_id	—
VEHICLE	vehicle_id	customer_id → CUSTOMER
PARKING_SLOT	slot_id	—
PARKING_SESSION	session_id	vehicle_id → VEHICLE; slot_id → PARKING_SLOT
BILL	bill_id	session_id → PARKING_SESSION
PAYMENT	payment_id	bill_id → BILL

7.2 Normalization

The relations are structured to avoid repeating groups and unnecessary duplication. Each table has a single primary key, so there are no partial dependencies. Non-key attributes describe the entity identified by the primary key, supporting Third Normal Form (3NF). Customer details are stored only in CUSTOMER, vehicle details only in VEHICLE, session details only in PARKING_SESSION, and financial details are separated into BILL and PAYMENT.

Normal Form	Applied Principle
1NF	Atomic attributes and no repeating groups.
2NF	Single-column primary keys remove partial dependency concerns.
3NF	Non-key attributes depend on the key and are separated by entity.

8. Data Dictionary

8.1 CUSTOMER

Column	Type	Constraint / Meaning
customer_id	INT	PK; customer identifier
name	VARCHAR(100)	Customer name
phone	VARCHAR(15)	Contact number
email	VARCHAR(100)	Email address

8.2 VEHICLE

Column	Type	Constraint / Meaning
vehicle_id	INT	PK
customer_id	INT	FK → customer
license_plate	VARCHAR(20)	Vehicle registration
vehicle_type	VARCHAR(50)	Car / Bike

8.3 PARKING_SLOT

Column	Type	Constraint / Meaning
slot_id	INT	PK
slot_number	VARCHAR(10)	Parking slot label
slot_type	VARCHAR / ENUM	Car / Bike slot type
status	ENUM	FREE / OCCUPIED

8.4 PARKING_SESSION

Column	Type	Constraint / Meaning
session_id	INT	PK
vehicle_id	INT	FK → vehicle
slot_id	INT	FK → parking_slot
entry_time	DATETIME	Entry timestamp
exit_time	DATETIME	Exit timestamp; NULL while active
status	ENUM	ACTIVE / COMPLETED

8.5 BILL

Column	Type	Constraint / Meaning
bill_id	INT	PK
session_id	INT	FK → parking_session
amount	DECIMAL(10,2)	Parking charge
bill_time	DATETIME	Billing timestamp

8.6 PAYMENT

Column	Type	Constraint / Meaning
payment_id	INT	PK
bill_id	INT	FK → bill
payment_time	DATETIME	Payment timestamp
amount_paid	DECIMAL(10,2)	Paid amount
payment_mode	ENUM	UPI / CARD / CASH
status	ENUM	SUCCESS / PENDING / FAILED

9. SQL Commands Used (DDL and DML)

9.1 DDL

Representative DDL used to define the relational structure:

```
CREATE DATABASE smart_parking_db;
USE smart_parking_db;

CREATE TABLE customer (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    phone VARCHAR(15),
    email VARCHAR(100)
);

CREATE TABLE vehicle (
    vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    license_plate VARCHAR(20),
    vehicle_type VARCHAR(50),
```

```
FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);
```

9.2 DML

```
INSERT INTO customer (name, phone, email)
VALUES ('Anjali Nair', '9876543219', 'anjali.nair@example.com');

UPDATE parking_slot
SET status = 'OCCUPIED'
WHERE slot_id = 12;

DELETE FROM customer
WHERE customer_id = 20;
```

9.3 Sample Output

Operation	Result
INSERT	1 customer record inserted
UPDATE	Parking slot status changed to OCCUPIED
DELETE	Selected customer record removed after dependency validation

10. Queries with Outputs

10.1 View all tables

```
SHOW TABLES;
```

Expected Output
customer, vehicle, parking_slot, parking_session, bill, payment

10.2 View occupied slots

```
SELECT slot_id, slot_number, slot_type, status
FROM parking_slot
WHERE status = 'OCCUPIED';
```

slot_id	slot_number	slot_type	status
2	A02	Car	OCCUPIED
3	B01	Bike	OCCUPIED

10.3 Active parking sessions

```
SELECT s.session_id, v.license_plate, ps.slot_number,
       s.entry_time, s.status
FROM parking_session s
JOIN vehicle v ON v.vehicle_id = s.vehicle_id
JOIN parking_slot ps ON ps.slot_id = s.slot_id
WHERE s.status = 'ACTIVE';
```

session_id	license_plate	slot_number	status
2	TS10CD5678	B01	ACTIVE
3	TS11EF9012	A02	ACTIVE

10.4 Presentation-II query — Anjali Nair

This query retrieves only the requested customer name, vehicle license plate, parking status, total parking duration, amount paid, and payment status.

```
SELECT c.name,
       v.license_plate,
       s.status AS parking_status,
       TIMESTAMPDIFF(MINUTE, s.entry_time,
                     COALESCE(s.exit_time, NOW())) AS parking_minutes,
       p.amount_paid,
       CASE UPPER(p.status)
         WHEN 'SUCCESS' THEN 'PAID'
         ELSE UPPER(p.status)
       END AS payment_status
FROM customer c
JOIN vehicle v ON v.customer_id = c.customer_id
JOIN parking_session s ON s.vehicle_id = v.vehicle_id
LEFT JOIN bill b ON b.session_id = s.session_id
LEFT JOIN payment p ON p.bill_id = b.bill_id
WHERE c.name = 'Anjali Nair';
```

Name	License Plate	Parking Status	Parking Time	Amount Paid	Payment Status
Anjali Nair	TS19UV8901	COMPLETED	120 minutes	₹120.00	PAID

10.5 Customer and vehicle relationship

```
SELECT c.name, v.license_plate, v.vehicle_type
FROM customer c
JOIN vehicle v ON c.customer_id = v.customer_id
ORDER BY c.customer_id;
```

Output: Customer names are displayed with their linked vehicle registration and vehicle type.

10.6 Total revenue from successful payments

```
SELECT SUM(amount_paid) AS total_revenue
FROM payment
WHERE status = 'SUCCESS';
```

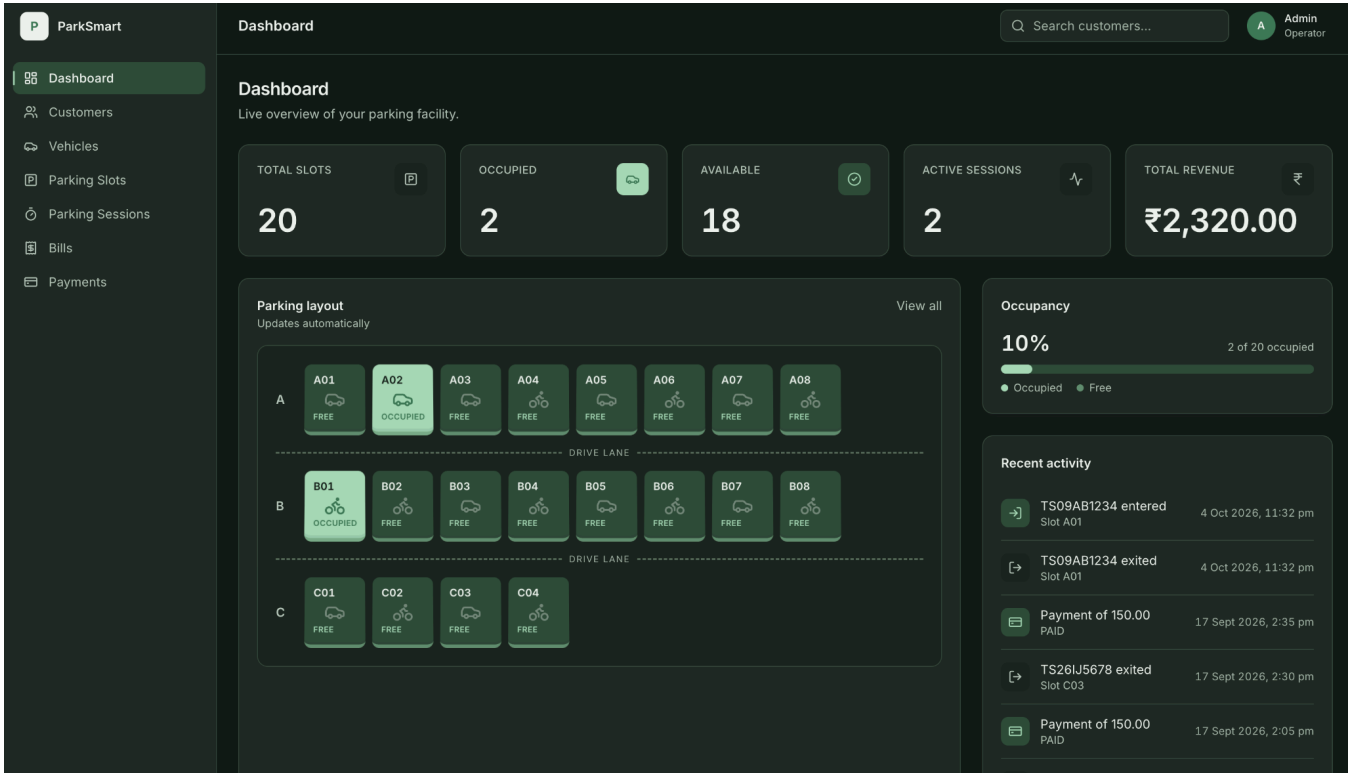
Output: A single aggregate value representing revenue from successful payments.

11. UI Design and Screenshots

The interface is designed as a focused parking operations dashboard. It provides dashboard statistics, parking slot visibility, customer and vehicle management, parking sessions, bills, and payments. The interface supports search, CRUD actions, confirmation dialogs, loading states, and error handling.

11.1 Screenshot placeholders

Dashboard Screenshot



Customer Management Screenshot

P

ParkSmart

Dashboard

Customers

Vehicles

Parking Slots

Parking Sessions

Bills

Payments

Dark mode

Customers

Search customers...

Admin Operator

Customers

Click a row to see vehicles and parking history.

Search...

All fields

+ Add Customer

Customer ID	Name	CONTACT	EMAIL	ACTIONS
1	Mithesh Kosanam	9876543210	mithesh@example.com	Edit Delete
2	Rahul Sharma	9876543211	rahul@example.com	Edit Delete
3	Ananya Reddy	9876543212	ananya@example.com	Edit Delete
4	Arjun Mehta	9876543213	arjun.mehta@example.com	Edit Delete
5	Sneha Reddy	9876543214	sneha.reddy@example.com	Edit Delete
6	Karthik Rao	9876543215	karthik.rao@example.com	Edit Delete
7	Priya Sharma	9876543216	priya.sharma@example.com	Edit Delete
8	Rohan Verma	9876543217	rohan.verma@example.com	Edit Delete
9	Neha Kapoor	9876543218	neha.kapoor@example.com	Edit Delete

1-10 of 19

< 1 / 2 >

Vehicle Management Screenshot

P

ParkSmart

Dashboard

Customers

Vehicles

Parking Slots

Parking Sessions

Bills

Payments

Dark mode

Vehicles

Search customers...

Admin Operator

Vehicles

Registered vehicles linked to customers.

Plate or customer...

All Car Bike

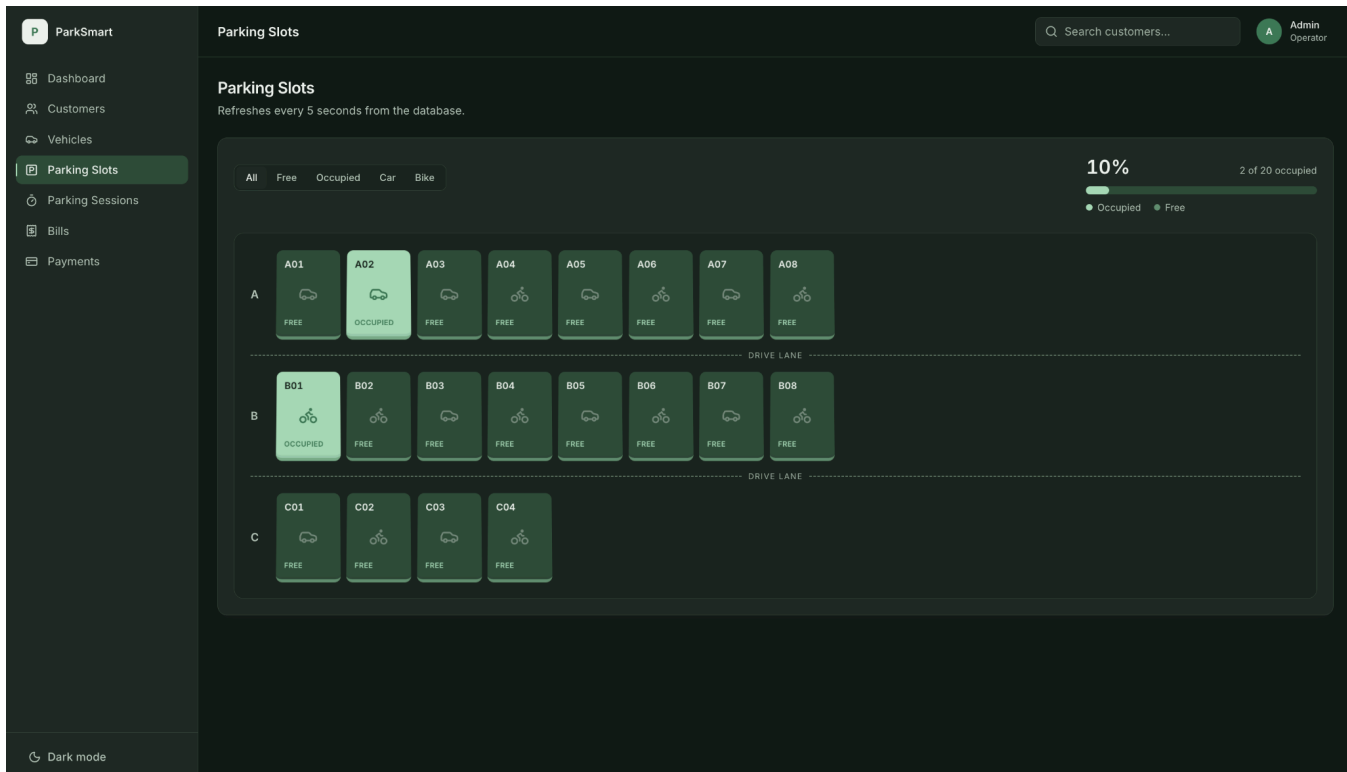
+ Add Vehicle

Vehicle ID	Customer	LICENSE PLATE	TYPE	ACTIONS
1	Mithesh Kosanam	TS09AB1234	CAR	Edit Delete
2	Rahul Sharma	TS10CD5678	BIKE	Edit Delete
3	Ananya Reddy	TS11EF9012	CAR	Edit Delete
4	Arjun Mehta	TS12GH1234	CAR	Edit Delete
5	Sneha Reddy	TS13IJ2345	BIKE	Edit Delete
6	Karthik Rao	TS14KL3456	CAR	Edit Delete
7	Priya Sharma	TS15MN4567	BIKE	Edit Delete
8	Rohan Verma	TS16OP5678	CAR	Edit Delete
9	Neha Kapoor	TS17QR6789	BIKE	Edit Delete

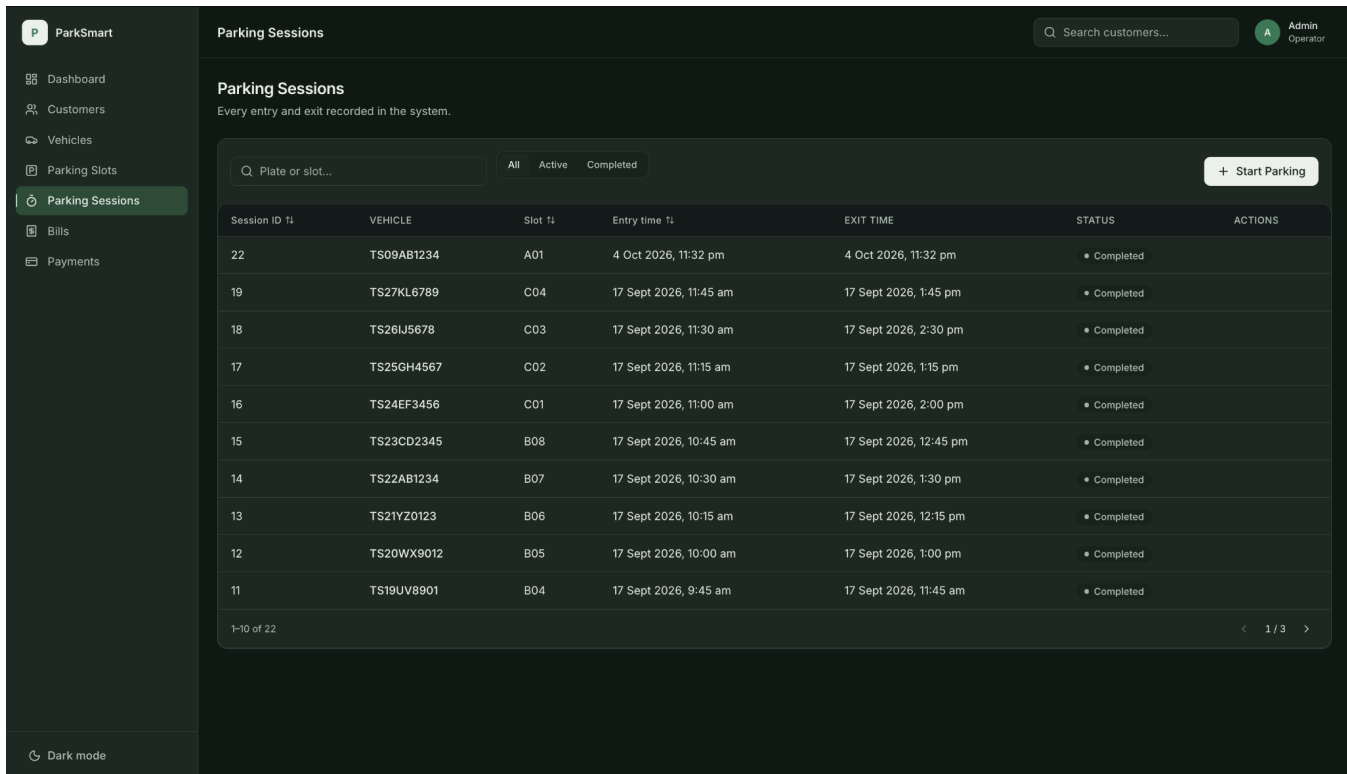
1-10 of 19

< 1 / 2 >

Parking slot Management Screenshot



Parking session Management Screenshot



Bills- Management Screenshot

P ParkSmart

Dashboard

Customers

Vehicles

Parking Slots

Parking Sessions

Bills

Payments

Dark mode

Bills

Charges raised for each parking session.

Q Bill ID, session ID or plate...

Bill ID ¹	SESSION ID	BILL INFORMATION	Amount ¹
19	19	TS27KL6789 · C04 · 17 Sept 2026, 1:45 pm	₹120.00
18	18	TS26JU5678 · C03 · 17 Sept 2026, 2:30 pm	₹150.00
17	17	TS25GH4567 · C02 · 17 Sept 2026, 1:15 pm	₹120.00
16	16	TS24EF3456 · C01 · 17 Sept 2026, 2:00 pm	₹150.00
15	15	TS23CD2345 · B08 · 17 Sept 2026, 12:45 pm	₹120.00
14	14	TS22AB1234 · B07 · 17 Sept 2026, 1:30 pm	₹150.00
13	13	TS21YZ0123 · B06 · 17 Sept 2026, 12:15 pm	₹120.00
12	12	TS20WX9012 · B05 · 17 Sept 2026, 1:00 pm	₹150.00
11	11	TS19UV8901 · B04 · 17 Sept 2026, 11:45 am	₹120.00
10	10	TS18ST7890 · B03 · 17 Sept 2026, 12:30 pm	₹150.00

1-10 of 19< 1/2 >

Payments- Management Screenshot

P ParkSmart

Dashboard

Customers

Vehicles

Parking Slots

Parking Sessions

Bills

Payments

Dark mode

Payments

Received, pending and failed payments.

All Paid Pending Failed

Payment ID ¹	BILL ID	Amount paid ¹	STATUS	Payment time ¹
19	19	₹120.00	Paid	17 Sept 2026, 1:50 pm
18	18	₹150.00	Paid	17 Sept 2026, 2:35 pm
17	17	₹120.00	Paid	17 Sept 2026, 1:20 pm
16	16	₹150.00	Paid	17 Sept 2026, 2:05 pm
15	15	₹120.00	Paid	17 Sept 2026, 12:50 pm
14	14	₹150.00	Paid	17 Sept 2026, 1:35 pm
13	13	₹120.00	Paid	17 Sept 2026, 12:20 pm
12	12	₹150.00	Paid	17 Sept 2026, 1:05 pm
11	11	₹120.00	Paid	17 Sept 2026, 11:50 am
10	10	₹150.00	Paid	17 Sept 2026, 12:35 pm

1-10 of 19< 1/2 >

12. Implementation Details

12.1 Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + TypeScript	Interactive user interface
Routing	TanStack Start + Router	Application routing
Data Fetching	TanStack Query	API requests and cache/refetch
Styling	Tailwind CSS v4	Responsive UI styling
Charts	Recharts	Dashboard visualization
Icons	Lucide React	Interface icons
Backend	Node.js + Express.js	REST API and business logic
Database Driver	MySQL2	MySQL connectivity
Database	MySQL	Persistent relational data

12.2 Database Connectivity

The backend loads environment configuration using dotenv and uses a MySQL connection pool. The Express server enables JSON requests and controlled CORS. The repository uses port 4000 for the backend and the frontend is configured to call the backend through VITE_API_BASE_URL, defaulting to http://localhost:4000.w

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=smart_parking_db
CORS_ORIGIN=http://localhost:8080
```

12.3 Key Backend Logic

```
const [rows] = await pool.query(
  "SELECT customer_id AS customerId, name, phone AS contact, email FROM customer"
);
```

Parameterized queries are used for user-controlled values. Parking session creation and completion use MySQL transactions so the session and slot status are changed together.

```
await conn.beginTransaction();

INSERT INTO parking_session ...
UPDATE parking_slot SET status = 'OCCUPIED' ...

await conn.commit();
```

```
// On failure:  
await conn.rollback();
```

12.4 API Structure

Resource	Main Endpoints
Customers	/api/customers, /api/customers/:id
Vehicles	/api/vehicles, /api/vehicles/:id
Slots	/api/slots
Sessions	/api/sessions, /api/sessions/:id
Bills	/api/bills, /api/bills/:id
Payments	/api/payments, /api/payments/:id
Dashboard	/api/stats, /api/activity
Search	/api/search/customer

13. Testing

Testing focused on database connectivity, CRUD operations, relational queries, parking-state consistency, and frontend-backend communication.

Test Case	Expected Result	Result
Database connection	Backend connects to smart_parking_db	Pass
View customers	Customer records load in UI	Pass
View vehicles	Vehicle records load with customer relationship	Pass
View slots	FREE/OCCUPIED states display correctly	Pass
Start session	Session becomes ACTIVE and slot OCCUPIED	Pass
Complete session	Session becomes COMPLETED and slot FREE	Pass
Customer search	Matching customer parking/payment data returned	Pass
Insert record	UI insertion persists in MySQL	Pass
Delete record	Valid deletion persists in MySQL	Pass

Test Case	Expected Result	Result
Dashboard statistics	Values reflect database state	Pass
Error handling	Invalid operations return controlled errors	Pass

14. Conclusion and Future Enhancements

14.1 Conclusion

The Smart parking lot management system provides a complete relational solution for managing parking operations. The six-table database separates operational and financial data, while foreign keys preserve relationships. The connected frontend and Express backend provide practical CRUD access, parking-session control, billing, payment tracking, search, and dashboard reporting.

14.2 Future Enhancements

- Authentication and role-based access control.
- IoT sensor integration for automatic slot detection.
- Online payment gateway integration.
- Notifications for active sessions and payment status.
- Cloud deployment and remote database access.
- Advanced analytics for occupancy and peak-hour patterns.

15. References

- MySQL Documentation — relational database, SQL, constraints and transactions.
- Node.js Documentation — backend runtime environment.
- Express.js Documentation — REST API framework.
- React Documentation — frontend component architecture.
- TanStack Query Documentation — server-state management and data fetching.
- Project GitHub Repository: <https://github.com/mitheshhhh/DBMS-Course-Project>

16. Appendix: GitHub Repository

The complete project source code, frontend, backend structure, startup scripts, and project documentation are maintained in the following repository:

<https://github.com/mitheshhhh/DBMS-Course-Project>

The repository contains the React/TypeScript frontend, Express/Node.js backend, MySQL integration, route and controller structure, environment configuration, and project startup scripts.